VC Generation

References And Borrowing

Goal

Extend Tiny TAC with references without adding a new VCGen core.

Plan:

- say what borrow commands **mean** as executions
- find the one place where that meaning fights SSA
- solve it with a value chosen from the future — a **prophecy**
- lower borrows to ordinary Tiny TAC and reuse the existing VCGen

The Puzzle This Deck Solves

```
entry:
  M := havoc
  i := havoc
  r, M2 := borrow_mut M[i]
  r2 := put_ref r, 7
  release r2
  x := M2[i]
  ok := x == 7
  assert ok
  halt
```

`borrow_mut` returns the reference **and** the continuation memory `M2` — in SSA, before any write through `r` has happened.

`x` is read from `M2`, which was created **before** the `7` was written. Why should `assert ok` hold?

By the end of the deck this will be obvious.

Reference Types

Tiny TAC gains:

$$\tau ::= \text{bool} \mid \text{int} \mid \text{bytemap} \mid \&\ \tau \mid \&\text{mut}\ \tau$$

The main case is a reference to an integer cell:

$$r : \&\text{int} \quad q : \&\text{mut int}$$

Borrow Commands

```
r := borrow M[i]
r, M2 := borrow_mut M[i]
q := borrow_ref r
q, r2 := borrow_ref_mut r
x := get_ref r
r2 := put_ref r, v
release r
```

Mutable borrowing returns both the reference and a continuation memory. Mutable reborrowing returns the child reference and the resumed parent reference.

That is the syntax. Before any encoding: what do these commands **do**?

What Borrowing Means

Read `r, M2 := borrow_mut M[i]` as a **loan** of slot `i`:

- while the loan lives, `r` is the only way to touch slot `i`
- `get_ref r` reads the loaned view; `put_ref r, v` overwrites it (returning a fresh reference, to stay in SSA)
- `release` ends the loan: the **last value written** through the reference is committed back to memory
- `M2` names the memory **after the loan ends** — the rest of the program reads memory through `M2`

> `M2` is **named** at borrow time, but its content at `i` is **determined** at release time.

That temporal gap is the entire difficulty of this deck.

The Puzzle, Executed

Run the loan semantics forward over the puzzle program:

after command	view through the borrow	M2[i]
`r, M2 := borrow_mut M[i]`	`r` sees `M[i]`	**?** — committed at release
`r2 := put_ref r, 7`	`r2` sees `7`	**?**
`release r2`	loan ends, `7` is committed	`7`
`x := M2[i]`	—	`7`, so `x = 7` and `ok` holds

Execution is fine: time flows forward, and the **?** resolves before anyone reads it. But an SSA definition of `M2` cannot wait — it needs a value for slot `i` **at borrow time**.

A Guess That Must Come True

Forget references. Tiny TAC can already talk about the future:

```
entry:
  guess := havoc
  double := guess * 2
  answer := 7
  assume guess == answer
  ok := double == 14
  assert ok
```

`double` is computed from `guess` **before** `answer` exists.

Is `assert ok` provable? `guess` is an arbitrary integer…

A Guess That Must Come True

Forget references. Tiny TAC can already talk about the future:

```
entry:
  guess := havoc
  double := guess * 2
  answer := 7
  assume guess == answer
  ok := double == 14
  assert ok
```

`double` is computed from `guess` **before** `answer` exists.

Is `assert ok` provable? `guess` is an arbitrary integer…

Yes — UNSAT. The `assume` discards every run whose guess was wrong. In all surviving runs `guess = 7`, so `double = 14` — as if `double` had been computed from the future value all along.

Why The Guess Trick Is Sound

- **havoc** = fork one run per possible future value of `guess`
- **assume** = at the moment the future arrives, kill every run where the guess disagrees
- for each real execution, **exactly one** guess survives — the correct one

So the program with (`havoc` now + `assume` later) has **the same observable behaviors** as one where the value magically arrived early: no behavior lost, none invented.

This is a **prophecy variable**. The formula is timeless — a constraint placed late reaches every use placed early. Only forward execution finds prophecies strange (it would have to guess).

Reference Triple

Now the borrow encoding writes itself. A reference is a triple:

$$r = \{\text{addr} : i, \text{value} : v, \text{promise} : p\}$$

- `addr`: the loaned location
- `value`: what the reference **currently sees**
- `promise`: the prophecy — a guess of the value the loan will commit at release

`promise` is exactly the `guess` from the previous slide, one per borrow: havoc'd when the loan starts, settled when it ends. Constant references never use theirs, but one shape keeps the lowering uniform.

Lowering: Direct Borrows

Constant borrow:

```
r := borrow M[i]
```

lowers to:

```
r := { addr: i,
       value: M[i],
       promise: havoc }
```

Mutable borrow:

```
r, M2 := borrow_mut M[i]
```

lowers to:

```
r := { addr: i,
       value: M[i],
       promise: havoc }
M2 := M[i := r.promise]
```

The continuation memory is defined **now**, at slot `i`, with the prophecy — the SSA gap is closed by the guess.

Lowering: Use And Release

Read:

```
x := get_ref r
```

lowers to:

```
x := r.value
```

Write:

```
r2 := put_ref r, v
```

lowers to:

```
r2 := { addr: r.addr,
        value: v,
        promise: r.promise }
```

Release:

```
assume r.value == r.promise
```

`release` is the `assume` of the guess trick: the value observed at the end of the loan **is** the promised value.

Original And Desugared, Side By Side

Original:

```
entry:
  M := havoc
  i := havoc
  r, M2 := borrow_mut M[i]
  r2 := put_ref r, 7
  release r2
  x := M2[i]
  ok := x == 7
  assert ok
```

Desugared:

```
entry:
  M := havoc
  i := havoc
  r := { addr: i, value: M[i],
         promise: havoc }
  M2 := M[i := r.promise]
  r2 := { addr: r.addr, value: 7,
          promise: r.promise }
  assume r2.value == r2.promise
  x := M2[i]
  ok := x == 7
  assert ok
```

The borrow becomes three facts: observe the current value, havoc the promise, and store the promise into the continuation memory — the future is written into `M2` on day one.

Original And Desugared, Side By Side

Original:

```
entry:
  M := havoc
  i := havoc
  r, M2 := borrow_mut M[i]
  r2 := put_ref r, 7
  release r2
  x := M2[i]
  ok := x == 7
  assert ok
```

Desugared:

```
entry:
  M := havoc
  i := havoc
  r := { addr: i, value: M[i],
         promise: havoc }
  M2 := M[i := r.promise]
  r2 := { addr: r.addr, value: 7,
          promise: r.promise }
  assume r2.value == r2.promise
  x := M2[i]
  ok := x == 7
  assert ok
```

The write produces a fresh view whose `value` is `7`. The promise rides along unchanged — the guess is about the **final** committed value, whoever writes it.

Original And Desugared, Side By Side

Original:

```
entry:
  M := havoc
  i := havoc
  r, M2 := borrow_mut M[i]
  r2 := put_ref r, 7
  release r2
  x := M2[i]
  ok := x == 7
  assert ok
```

Desugared:

```
entry:
  M := havoc
  i := havoc
  r := { addr: i, value: M[i],
         promise: havoc }
  M2 := M[i := r.promise]
  r2 := { addr: r.addr, value: 7,
          promise: r.promise }
  assume r2.value == r2.promise
  x := M2[i]
  ok := x == 7
  assert ok
```

The release settles the guess: the last observed value equals the promise. From here on, every run that survives has `r.promise = 7`.

Original And Desugared, Side By Side

Original:

```
entry:
  M := havoc
  i := havoc
  r, M2 := borrow_mut M[i]
  r2 := put_ref r, 7
  release r2
  x := M2[i]
  ok := x == 7
  assert ok
```

Desugared:

```
entry:
  M := havoc
  i := havoc
  r := { addr: i, value: M[i],
         promise: havoc }
  M2 := M[i := r.promise]
  r2 := { addr: r.addr, value: 7,
          promise: r.promise }
  assume r2.value == r2.promise
  x := M2[i]
  ok := x == 7
  assert ok
```

The tail is untouched. `M2[i]` already contains the — now settled — promise, so `x = 7`. Only ordinary assignments, a store, and an assume remain: existing VCGen applies unchanged.

Why It Proves

$$
\begin{aligned}
\mathrm{M2} &= M[i := \mathrm{promise}(r)] \\
\mathrm{value}(\mathrm{r2}) &= 7 \\
\mathrm{promise}(\mathrm{r2}) &= \mathrm{promise}(r) \\
\mathrm{value}(\mathrm{r2}) &= \mathrm{promise}(\mathrm{r2})
\end{aligned}
$$

Why It Proves

$$\begin{aligned} \mathrm{M2} &= M[i := \mathrm{promise}(r)] \\ \mathrm{value}(\mathrm{r2}) &= 7 \\ \mathrm{promise}(\mathrm{r2}) &= \mathrm{promise}(r) \\ \mathrm{value}(\mathrm{r2}) &= \mathrm{promise}(\mathrm{r2}) \end{aligned}$$

Chaining the last three equalities:

$$\mathrm{promise}(r) = 7$$

So `M2` — written back at borrow time — stores `7` at address `i`, and `x == 7` follows. The havoc'd promise was **forced** by the release assumption.

Demo: The Whole Pipeline

Desugaring is a `ttac -> ttac` pass; the VC core never sees a reference:

```
$ ttac desugar safe_borrow_mut.ttac | ttac vcgen - --solve
unsat
$ ttac desugar unsafe_borrow_mut.ttac | ttac vcgen - --solve
sat
```

Demo: The Whole Pipeline

Desugaring is a `ttac -> ttac` pass; the VC core never sees a reference:

```
$ ttac desugar safe_borrow_mut.ttac | ttac vcgen - --solve
unsat
$ ttac desugar unsafe_borrow_mut.ttac | ttac vcgen - --solve
sat
```

And the prediction from the guess-trick slide — forward execution **cannot** run a borrow program:

```
$ ttac run safe_borrow_mut.ttac
status: stopped (assume failed in block entry)
```

The interpreter picks a value for the havoc'd promise and dies at the release `assume`. Only the solver — choosing all values at once — threads the prophecy.

Mutable Reborrow

```
r, M2 := borrow_mut M[i]
q, r2 := borrow_ref_mut r
q2 := put_ref q, 7
release q2
r3 := put_ref r2, 8
release r3
```

`q` borrows through `r`: a loan of a loan. While `q` lives, `r` is suspended; after `q` is released, `r2` resumes the parent reference and can overwrite the final promised value.

Reborrow Lowering

```
q := { addr: r.addr, value: r.value, promise: havoc }
r2 := { addr: r.addr, value: q.promise, promise: r.promise }
```

Two prophecies now: `q.promise` guesses what the **child** loan commits; the parent's resumed view `r2` starts at exactly that guess. `r2` keeps the original promise — the outer loan still owes memory its final value.

Releasing `q` settles the inner guess; releasing `r2` settles the outer one.

Reborrow Facts

$$\begin{aligned}
\mathrm{M2} &= M[i := \mathrm{promise}(r)] \\
\mathrm{value}(\mathrm{q2}) &= 7 \\
\mathrm{value}(\mathrm{q2}) &= \mathrm{promise}(q) \\
\mathrm{value}(\mathrm{r3}) &= 8 \\
\mathrm{promise}(\mathrm{r3}) &= \mathrm{promise}(\mathrm{r2}) \\
\mathrm{promise}(\mathrm{r2}) &= \mathrm{promise}(r)
\end{aligned}$$

The inner release forces `promise(q) = 7`, so `r2` resumes seeing `7`. The outer release forces `promise(r) = 8` — the parent's final write wins, and `M2[i] == 8`.

Prophecy Variables: The Name And The History

What we built has a name and a literature:

- Abadi and Lamport, *The Existence of Refinement Mappings* (1991) — prophecy variables: auxiliary values chosen nondeterministically now, constrained by the future, without changing observable behavior.
- Matsushita, Tsukada, Kobayashi, *RustHorn: CHC-based Verification for Rust Programs* (2020) — a mutable borrow **is** a (current, final) value pair; no memory model needed for safe Rust.
- Priya and Gurfinkel, *Ownership in low-level intermediate representation* (FMCAD 2024) — the same idea in a BMC setting, mixed with an address-map memory model. Tiny TAC follows this pattern.

The `promise` field is RustHorn's "final value", made to coexist with explicit bytemaps: the borrow writes it into the continuation memory, the release proves it right.

VCGen Boundary

After lowering, VCGen only sees ordinary Tiny TAC:

- assignments
- map reads and stores
- assumptions
- assertions
- havoc values

Borrow **well-formedness** (liveness, exclusivity, reborrow lifetimes) is a separate side condition — checked before VCGen, or encoded as extra obligations. Candidate disciplines: Stacked Borrows, Tree Borrows.

Open Extensions

- **Finite-region borrows** — borrow a byte range, not a single cell: base + size + per-word value/promise. Question: keeping the encoding compact.
- **Reference shadow memory** — store references in memory via ghostmap shadows for addr/value/promise/permission. Question: which metadata must be shadowed, and how shadows stay synchronized.
- **Reference semantics and well-formedness** — pick a borrow model (Stacked/Tree Borrows), state the rules, decide: reject before VCGen or encode as obligations.

Each is small enough to state independently, but substantial enough to be a useful project.